You will need transaction data in the production catalog to create the model. Note the new setup variable `$env=prod`. The pipeline workflow notebooks: `CH7-01, CH7-02, and CH7-03` are ready to be added to a workflow in the jobs section of Databricks. Start by clicking the **Workflows** icon on the left-side menu bar. Next, click **Create job**. We have provided you with a `YAML` file, `production streams.yaml`, to guide you. Notice, in the `YAML` and *Figure 7.14*, the tasks do not depend upon one another.

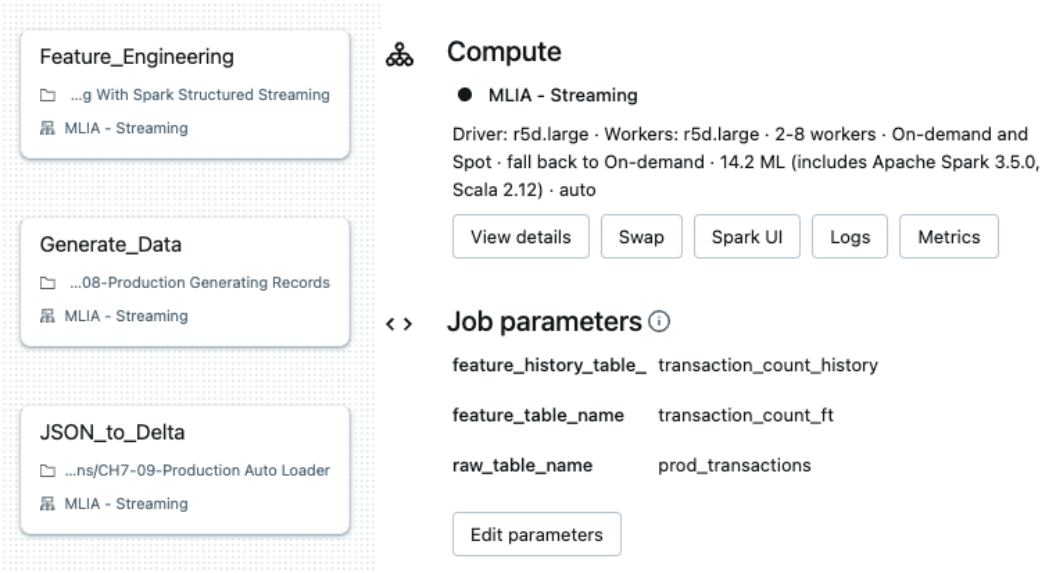


Figure 7.14 – The task DAG (L) and some settings (R) for the Production Streaming Transactions job

As shown in *Figure 7.14*, you can use the compute cluster shown in *Figure 7.6* for the job and the other notebooks. Also, we chose to add parameters at the job level rather than the individual tasks. You can now easily generate the data needed to build the model.

We will use the `training_set` created in *Chapter 6* for training a new version of the model. Recall that logging a model in Unity Catalog packages the feature metadata with the model. Hence, at inference time, it automatically looks up features from feature tables provided in the specified training set. We added this spec to both `Wrapping and Logging the Model` notebooks. We will not go through these notebooks separately. The difference between them is the baseline model is trained on data in the development catalog. Then the model is re-trained in production using production data from the inference table.

We know that we don't have an inference table yet. Don't worry—it's coming! Back to the `training_set`, being able to match up the features is only useful if we have feature values. We use the time boundaries of our feature table to ensure that the raw transactions being used for training have feature values. Also, we require the label column to be present, as shown in *Figure 7.15*.